

Relatório Técnico — Trabalho Integrador

Sistema de Controle de Estoque e Pedidos — Backend (API REST)

Universidade Federal da Fronteira Sul — Ciência da Computação GEX613 — Programação II Autor: **Eduardo Klassen** — Matrícula 20240017565 Data: junho de 2026

1. Introdução

Este relatório descreve o desenvolvimento do backend de um sistema de controle de estoque e pedidos, proposto como Trabalho Integrador da disciplina GEX613 — Programação II. O trabalho articula conhecimentos de três componentes curriculares: Programação II (construção da aplicação), Banco de Dados (modelagem relacional) e Curso pessoal Club Full-Stack que ensinou sobre básico de arquitetura e organização do código, principalmente em PHP em vez do uso de JavaScript que foi colocado em aula.

O objetivo foi construir uma API REST funcional que oferecesse operações CRUD sobre as entidades do domínio, autenticação e autorização de usuários, e a lógica de negócio de uma pequena empresa que adquire materiais, comercializa produtos e controla suas movimentações financeiras.

O projeto foi desenvolvido individualmente.

2. Descrição do sistema

O sistema modela o funcionamento de uma empresa de pequeno porte sob a ótica de três fluxos integrados:

- **Fluxo comercial:** clientes realizam pedidos, compostos por itens (produtos), com controle de quantidade e valores.
- **Fluxo de estoque:** a empresa adquire materiais de fornecedores por meio de compras; produtos são compostos por materiais.
- **Fluxo financeiro:** pedidos geram receitas e compras geram despesas, ambas refletidas em um caixa.

Esses fluxos se conectam: ao registrar uma compra, o estoque dos materiais aumenta; ao criar um pedido, o estoque dos produtos diminui. As movimentações financeiras acompanham essas operações.

3. Tecnologias utilizadas e justificativas

A escolha das tecnologias priorizou o aprendizado dos fundamentos e foi totalmente pessoal, pelo gosto por PHP.

PHP 8 com orientação a objetos, sem framework. A decisão de não adotar um framework (como Laravel) foi deliberada. Frameworks resolvem muitos problemas automaticamente — roteamento, ORM, autenticação — mas escondem como essas peças funcionam. Para esse trabalho, seguindo o curso de PHP Club Full-Stack, tentei construir manualmente o roteador, o middleware de autenticação e o acesso ao banco, o que proporciona um entendimento mais profundo.

MySQL/MariaDB como banco relacional. O domínio possui entidades fortemente relacionadas (pedidos pertencem a clientes, itens pertencem a pedidos, etc.), o que torna o modelo relacional adequado. O uso de chaves estrangeiras e constraints permite que o próprio banco garanta a integridade dos dados.

Autenticação por JWT (firebase/php-jwt). Optou-se por tokens JWT em vez de sessões tradicionais por serem *stateless*: o servidor não precisa armazenar estado de sessão, pois o próprio token carrega as

informações do usuário de forma assinada. Essa abordagem é adequada a uma API REST e facilita o consumo futuro por um front-end separado.

vlucas/phpdotenv para configuração. Mantém credenciais sensíveis (senha do banco, chave secreta do JWT) fora do código-fonte, em um arquivo `.env` que não é versionado.

4. Arquitetura da aplicação

A aplicação adota uma arquitetura em camadas, com separação clara de responsabilidades — princípio central da Engenharia de Software, ensinado por curso pessoal que aborda o básico de arquitetura para funcionamento e boas práticas.

O ponto de entrada é único (`public/index.php`): toda requisição passa por ele. Em seguida, o roteador (`app/Core/Router`) identifica qual controlador deve tratar a URL e o método HTTP. Antes de chegar ao controlador, requisições protegidas passam por *middlewares* que validam o token de autenticação e o perfil do usuário. O controlador valida os dados de entrada e delega o acesso ao banco ao *model* correspondente, que executa as consultas via PDO. A resposta é sempre padronizada em JSON por um *helper* dedicado.

Essa organização reflete o padrão de separação entre **roteamento, controle, modelo e apresentação**, tornando o código previsível: cada tipo de tarefa tem um lugar definido.

5. Modelagem do banco de dados

O modelo evoluiu de uma versão inicial para uma versão revisada, resultado de um processo de análise crítica. O modelo entidade-relacionamento foi **validado em consultoria com um profissional experiente em modelagem de dados**, cuja avaliação confirmou a coerência geral das cardinalidades e dos relacionamentos, e sugeriu melhorias pontuais que foram incorporadas. As principais decisões:

Usuário com perfil por atributo, não por entidade. Em vez de criar uma entidade separada para perfis de acesso, o tipo de usuário (`admin/comum`) é um atributo da própria tabela `usuario`. Como há apenas dois perfis fixos, uma entidade separada adicionaria complexidade sem benefício.

Ligação direta entre Usuário e Pedido (rastreabilidade). Originalmente, o pedido se ligava apenas ao cliente. Identificou-se que era importante registrar *qual usuário do sistema* lançou cada pedido, e não apenas a qual cliente ele pertence. Adicionou-se, portanto, uma chave estrangeira `id_usuario` em `pedido`, melhorando a rastreabilidade das operações.

Histórico de status do pedido. Criou-se a entidade `historico_status` para registrar cada transição de status de um pedido (por exemplo, de "aberto" para "pago"), guardando o status anterior, o novo e a data da mudança. Isso permite reconstruir toda a linha do tempo de um pedido — funcionalidade semelhante ao rastreamento de uma encomenda. A cada mudança de status, uma nova linha é inserida automaticamente.

Endereço como entidade própria. Em vez de tratar endereço como atributo multivalorado de cliente e fornecedor, modelou-se como uma entidade independente. Isso evita a colisão de identificadores que ocorreria caso se usasse um atributo multivalorado compartilhado, e permite que tanto clientes quanto fornecedores tenham endereços de forma consistente.

Relacionamento Fornecedor-Material via Compra. Avaliou-se inicialmente uma ligação direta (muitos-para-muitos) entre fornecedor e material. Concluiu-se que essa ligação seria redundante, pois o fornecedor de um material já é determinado pela compra na qual o material entrou. Removeu-se, portanto, o relacionamento direto, deixando o fornecedor associado à compra — uma decisão que elimina duplicação de informação e possibilidade de inconsistência.

Tabelas associativas para relacionamentos N:N. Dois relacionamentos muitos-para-muitos foram resolvidos com tabelas associativas: `compra_material` (uma compra contém vários materiais, cada um com quantidade e custo) e `produto_material` (um produto utiliza vários materiais em sua composição).

O modelo final possui **15 tabelas**, organizadas nos módulos de acesso, comercial, estoque, composição e financeiro.

6. Funcionalidades implementadas

- **Autenticação e autorização:** login que retorna um token JWT; verificação do token em rotas protegidas; restrição de operações de cadastro ao perfil administrador.
- **Operações CRUD** sobre todas as entidades principais.
- **Controle de estoque automático:** ao registrar uma compra, as quantidades são somadas ao estoque dos materiais; ao criar um pedido, são subtraídas do estoque dos produtos.
- **Histórico de status:** registro automático de cada mudança de status de pedido.
- **Uso de transações:** a criação de pedidos e compras — que envolve gravar a operação e seus itens, além de atualizar o estoque — é feita dentro de uma transação de banco de dados. Assim, ou todas as operações são efetivadas, ou nenhuma é, garantindo a consistência dos dados em caso de falha.
- **Dashboard:** endpoint que retorna contagens e um resumo dos dados.

7. Desafios encontrados

Durante o desenvolvimento houve diversos obstáculos técnicos, que contribuíram para o aprendizado:

Configuração do ambiente PHP. Ao executar o `composer install`, ocorreu um erro relativo à ausência da extensão `zip`. De forma semelhante, a primeira tentativa de login falhou por falta da extensão `pdo_mysql`. Ambos os problemas foram resolvidos habilitando as respectivas extensões no arquivo `php.ini`, o que exigiu compreender como o PHP carrega suas extensões.

Vulnerabilidade em dependência. O Composer bloqueou a instalação da versão inicialmente especificada da biblioteca de JWT (`firebase/php-jwt ^6.0`) por conta de um alerta de segurança conhecido. A solução foi atualizar para a versão `^7.0`, que corrige a vulnerabilidade — uma lição sobre a importância de manter dependências atualizadas.

Particularidades do terminal. No PowerShell (terminal padrão do ambiente de desenvolvimento), o operador de redirecionamento `<`, comumente usado para importar arquivos SQL, não é suportado. Foi necessário adaptar os comandos para usar `Get-Content`, e essa observação foi documentada no guia de execução para facilitar a avaliação.

Ordem de criação das tabelas. Por conta das chaves estrangeiras, as migrations precisaram ser executadas em uma ordem específica — por exemplo, a tabela de endereços depende da existência da tabela de fornecedores. A ordem correta foi estabelecida e documentada.

Erros de sintaxe e rotas. Por falta de prática, muitas incoerências ocorreram: verificação de sintaxe, ajuste e organização das rotas da API e correção da modelagem criada em BD1.

8. Testes

A API foi testada com a ferramenta **Insomnia**, por meio de uma coleção de requisições organizada por módulo. O processo de teste validou o ciclo completo: autenticação (obtenção e uso do token), operações

CRUD, a lógica de estoque (verificando que o estoque aumenta após uma compra e diminui após um pedido) e o registro do histórico de status (verificando que cada mudança gera uma nova linha no histórico).

9. Uso de ferramentas de inteligência artificial

Em conformidade com a orientação da disciplina, declara-se o uso de uma ferramenta de inteligência artificial (assistente Claude, da Anthropic) como apoio durante o desenvolvimento. Como estava sozinho e havia falta de conversas em equipe para testes e práticas, o uso foi necessário; deu-se de forma supervisionada, com todas as decisões de projeto, validações e testes conduzidos pelo autor.

- **Geração de código seguindo o padrão estabelecido:** a ferramenta auxiliou na escrita dos *models*, *controllers* e *migrations* das novas entidades (materiais, compras, receitas, despesas, endereços, caixa e histórico de status), mantendo a coerência com o código existente.
- **Revisão e correção:** verificação de sintaxe, ajuste e organização das rotas da API e atualização da coleção de testes do Insomnia para refletir as novas entidades.
- **Apoio na modelagem:** discussão das decisões do modelo entidade-relacionamento (a ligação direta entre usuário e pedido e o histórico de status), que foram posteriormente validadas em consultoria com um profissional de modelagem e decididas pelo autor.
- **Explicação de conceitos e documentação:** esclarecimento de conceitos técnicos e apoio na redação da documentação do projeto de modo mais detalhado para o leitor.

A ferramenta atuou como recurso de apoio, de forma análoga à consulta a documentação técnica e à consultoria com especialista.

10. Considerações finais

O desenvolvimento do backend permitiu aplicar, de forma integrada, conceitos de programação orientada a objetos, modelagem de banco de dados relacional e princípios de arquitetura de software. A opção por construir a aplicação sem um framework, embora mais trabalhosa, proporcionou compreensão aprofundada de mecanismos que normalmente são abstraídos, como roteamento, middleware de autenticação e controle transacional.

Repositório

Código-fonte: <https://github.com/Eduardoklassen/Projeto-Integrador-UFFS>